Open in app ↗ Sign up Sign in

Medium Search

ZipRecruiter Tech · Follow publication

# Multimodal Learning for Employment Marketplace Recommendation

17 min read · Jul 20, 2023

Engineering Team Follow

Listen Share

ZipRecruiter reveals its powerful ML matching architecture, suitable for most bi-directional recommendation needs.

ZipRecruiter has built one of the world's largest, online, double-sided job marketplaces. It enables job-seekers to find their next great career opportunity, and helps employers discover the best available candidates for their open positions. A key feature of our marketplace is making 'the search' as efficient as possible.

To that end, our recommendation team is responsible for actively generating optimal matches between job-seekers and jobs using advanced machine learning tools. The current system has increased job seeker engagement by 12% and boosted their satisfaction, with a similar effect for employers.

Here, we'll outline how challenges in our data and the nature of our marketplace influenced model development, and then dive into the loss-function and symmetrization intricacies of our unique learning architecture. Anyone building a recommender system, especially for two or more market sides that require recommendations, will potentially find this useful.

## Model requirements in light of 4 main challenges

Generating great matches requires overcoming substantial challenges related to the nature of our marketplace and the structure of the data.

### 1. Optimizing for real-time inference on big-data

Our marketplace consists of a large volume of entities which directly affect the way our models operate and perform. In fact, we have more than 36 million resumes and over 12 million active job postings to match. Job-seekers and employers generate a very large quantity of events describing the interactions between them containing hundreds of millions of applications, clicks, searches, hires, and rating signals per year. In terms of recommendation use cases, these numbers correspond to billions of potential entity pair matches.

At this massive scale, suggesting high-quality matches to our users requires an efficient and scalable inference mechanism to calculate a meaningful matching score for every pair of entities.

To illustrate this, let's consider the case of recommending the next best job to a given job-seeker. A naive solution may include an ML model that for each potential job receives job-seeker input data like resume, past activity etc., and produces a relevance score for it (higher relevance score corresponds to a better match). However, this design depends directly on the modeling choice and its associated computational complexity where the inference model is called with the entire job-seeker input.

Instead, we can optimize our model to act on an encoded entity representation in the form of an embedding in a semantic vector space thus enabling the use of a simple **scalar product calculation** and achieving scalable and more efficient real-time inference.

#### 2. Maintaining performance with extremely diverse and dynamic data

Another big challenge is related to the rich and non-stationary nature of the marketplace which includes seasonality patterns of job openings and diverse job titles. Our modeling solutions need to accommodate the high level of diversity inherent to the marketplace entities.

Data in real-world applications typically exhibit long-tailed distributions, where attributes are dominated by a small portion of values, while all the other values are associated with only a few samples each.

To better understand this, let's look at the "*job title*" attribute. Obviously, "*job title*" plays an extremely important role in representing the job entity. However, most titles are associated with only a fraction of the job-postings and thus relevant for only a small cohort of job-seekers. It is non-trivial to properly represent tens of thousands of unique job titles in the training process when each individually has a very low probability to appear in the data, thus potentially leading to poor model performance.

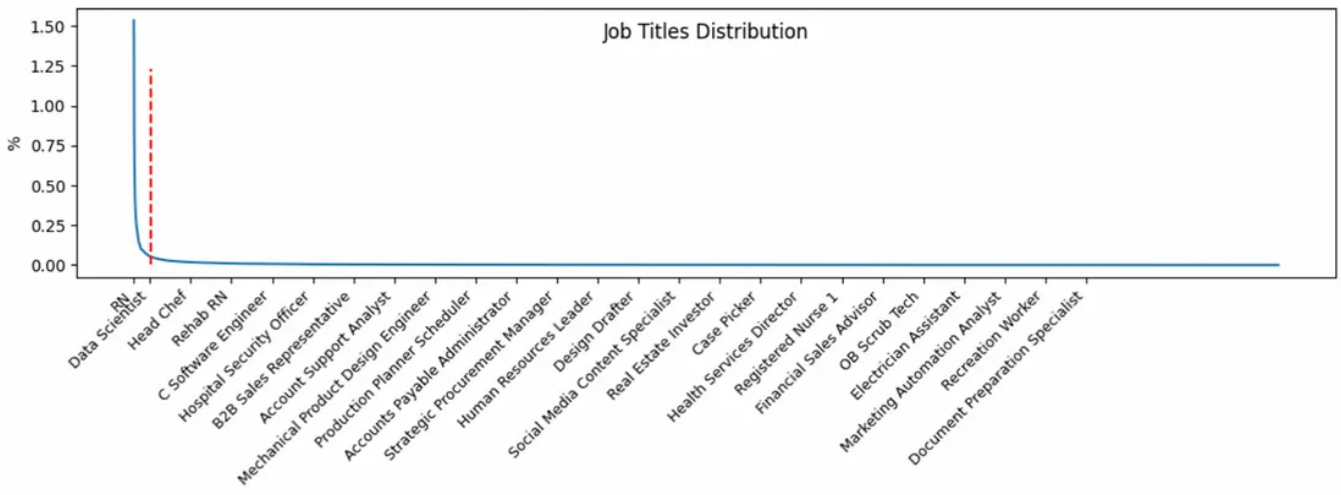


Figure 1: Most titles are rare and appear only within a fraction of the job listings. This makes the training of machine learning models extremely challenging.

In the chart above, “Data Scientist” jobs are roughly 0.1% of all available jobs. This means that collecting enough data and training a model in a way that all the attribute values are sufficiently represented becomes very difficult.

In addition, new entities are continuously generated and removed from the marketplace, creating a dynamic and evolving environment. As a result, our models need to constantly adapt to maintain their performance and accurate predictions. The following plot shows the amount of new jobs being added each day during a period of 3 months. Having a constant influx of new jobs implies that our model is required to perform accurate predictions for unseen classes at inference time.

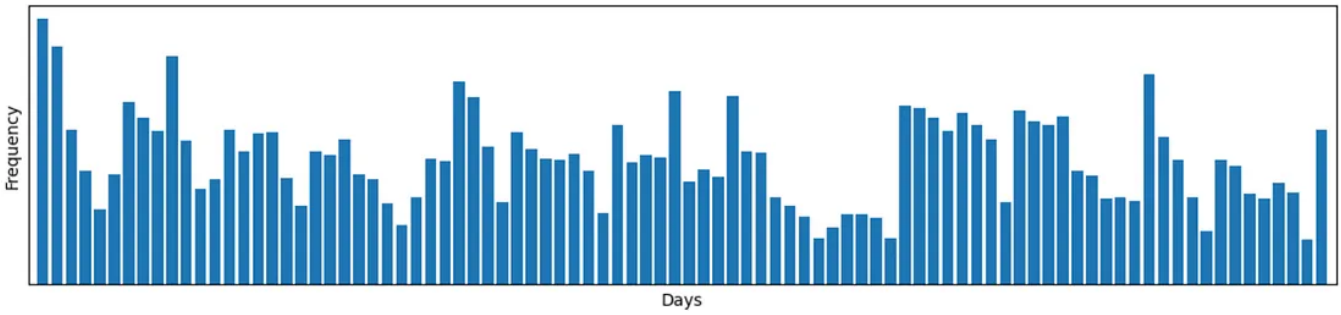


Figure 2: The number of new jobs added to the marketplace each day maintains its magnitude.

Poor entity representation during training and the high relative appearance of new entities during inference time dictates a need for **zero-shot** capabilities. This

essentially means that our model will be able to accurately represent new entities in the online environment without the need to explicitly see them during training.

#### 3. Multi-faceted complexity requires context-aware modeling

The nature of the data stems from the entities' representations in the marketplace. Both the job-seeker and the job posting are primarily unstructured text based and uniquely express their author's nuanced details, which frequently exhibits personal statements or taste. Large processing and modeling overhead is generated from the following data complexity characteristics:

- **Unstructured** — The entities are mostly **unstructured textual data** meaning that the data is designed for humans and not for machines. Visual perception plays an important role which results in a lack of order and common structure that could be utilized.

- **Non-standard format for recurring parameters** — In addition, entities do not follow specific design conventions or rules. For example, the format of the "skills level" may involve numbers, visual scale or simply a word.

- **Creative titles** — Employers often take the liberty to be creative and come up with improper language phrases, slang and even superlative adjectives as part of the full job title. We are all familiar with titles like — "Rockstar Java Developer" and "Ninja Sales Representative".

- **Domain specific jargon** — Certain professions make use of unique jargon which requires domain expertise, so it's not always possible to extract their inherent meaning. For example, the abbreviations in the nursing profession include — LPN, LVN, RN, NP and many many more.

- **Context aware terminology** — Similarly, some terms may receive their true meaning only within the context of their entity. For example, "MD" can represent both "Medical Doctor" and "Managing Director".

- **Diverse sources** — Our data originates from multiple sources, namely, job-seekers, employers and the marketplace itself. Job-seekers are mainly characterized by information parsed from their resume, like employment history, education, skills and more. Jobs, on the other hand, are characterized by

title and various attributes extracted from the job description. These include desired skills, education level, benefits, location and whether it is an in-person or remote job. Keep in mind, many of the displayed jobs are collected from the web, and as a result, we don't have access to the employer's original metadata pertaining to the job. For the marketplace, we have all the engagement data generated on our platform which represents the interactions between the job-seekers and the employers (e.g. searches, impressions, clicks, applications, interest signals, hiring signals…)

These complexities suggest a **context aware** modeling scheme. One relevant architecture is attained by constructing a **pre-trained encoder** that will provide a unique and independent representation for each entity type. As we shall see later on, this choice enables more diverse and flexible utilization in downstream modeling applications.

#### 4. Symmetrically balancing competing stakeholder objectives

Generating recommendations in a multi-sided marketplace introduces significant optimization challenges. These challenges naturally arise from the marketplace structure and the associated global business goals.

To get a sense of what this means, consider job recommendations that we provide to a job-seeker. Even though this action essentially faces one side of the marketplace, we cannot ignore the other side's objectives and how they are affected by them. For example, if we suggest a potentially relevant job to a candidate, even though we estimate that the employer is *un*likely to be interested, we might generate a wrong prioritization in which the less relevant candidates are recommended to the employer over relevant ones, and eventually affect the employers' overall satisfaction. In other words, the optimization of our recommendation models needs to take into account the various entity objectives, whereas, naively optimizing for only a single marketplace side might lead to greedy behavior and eventually to inevitably suboptimal performance. As we shall see, we are able to avoid this by leveraging other job-seekers' activity to form a more balanced job-seeker to job score matrix.

In addition to the obvious entities objectives, as the marketplace manager,

ZipRecruiter has its own metrics that require optimization which usually serve more global business goals. These can, for example, balance the marketplace or introduce fairness into our ranking mechanism. Fortunately, these objectives can sometimes be modeled separately from those “entity-facing” objectives.

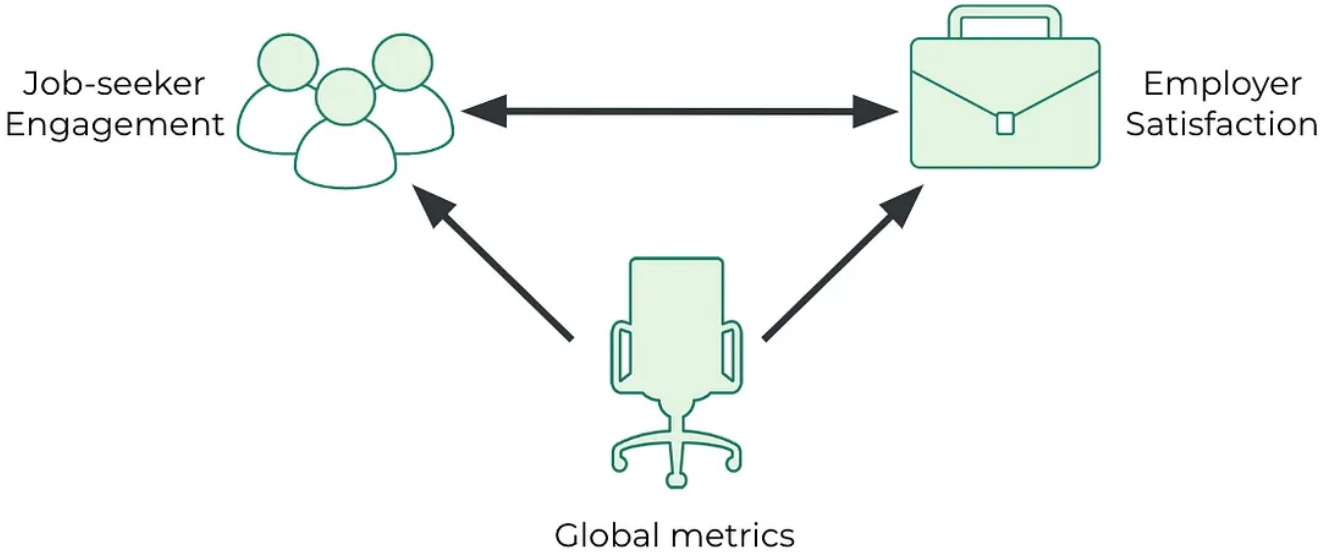


Figure 3: Multiple objectives in ZipRecruiter’s marketplace.

As suggested above, we approach this problem at the optimization level directly. In order to incorporate the desired behavior and, by design, achieve balance between the marketplace’s different entity-facing objectives, our **multimodal multi-task** model is optimized using a **self-regularized dual loss function**. As we shall see later on, this loss imposes symmetry considerations in the form of two-sided calibration of the two optimization tasks, which is necessary for producing a well behaved and calibrated model.

## Learning similarity and balancing with pre-trained encoders

For machine learning models, achieving good generalization depends highly on their ability to represent the data. By explicitly introducing relational inductive bias amongst the marketplace entities, based on their semantic similarity, we are able to impose constraints on their relationships and interactions in a learning process. This structure will hopefully enable the model to learn meaningful representations

and better capture same-entity as well as cross-entity semantic similarities.

**Pre-trained encoders for same-entity similarity learning**

In order to define what semantic similarity *within the same entity type* is, we can exploit the information encoded in the interactions between job-seekers and jobs. When a job-seeker reviews a job and decides that it is relevant for them, they can go ahead and apply for it.

Considering all the relevant jobs that the job seeker applied to, we can safely assume that these jobs are similar to each other in their meaning to that specific job-seeker. This similarity concept provides us valuable information and can be used for learning an appropriate job representation metric. While similar jobs should be close to each other, non-similar jobs should be far away from each other.

The same idea can also be applied in the case of two job-seekers who apply to a given job. In this scenario, the two job-seekers who received a positive rating (thumbs-up) from the employer are considered similar and should be close. However, unlike the job similarity case, employer feedback can also provide explicit negative feedback in which job-seekers can receive a negative rating (thumbs-down). As we shall see later on, this has an implication on the modeling choice for each case.

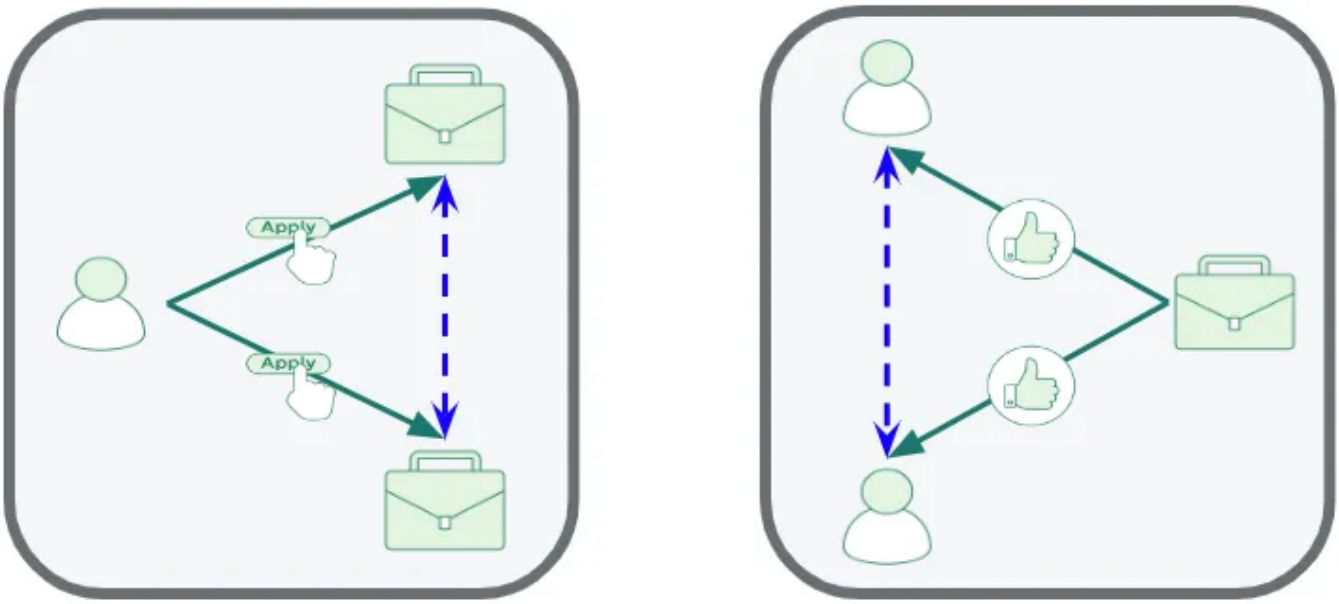


Figure 4: Exploiting the implicit semantic structure of the interactions between the entities, we are able to construct an entity self-similarity framework and have a consistent way of learning their latent

representations.

This entity self-similarity framework essentially provides a way to project the learned semantic similarity and use it as a solid building block for modeling cross entity relations and performing inference on unseen instances.

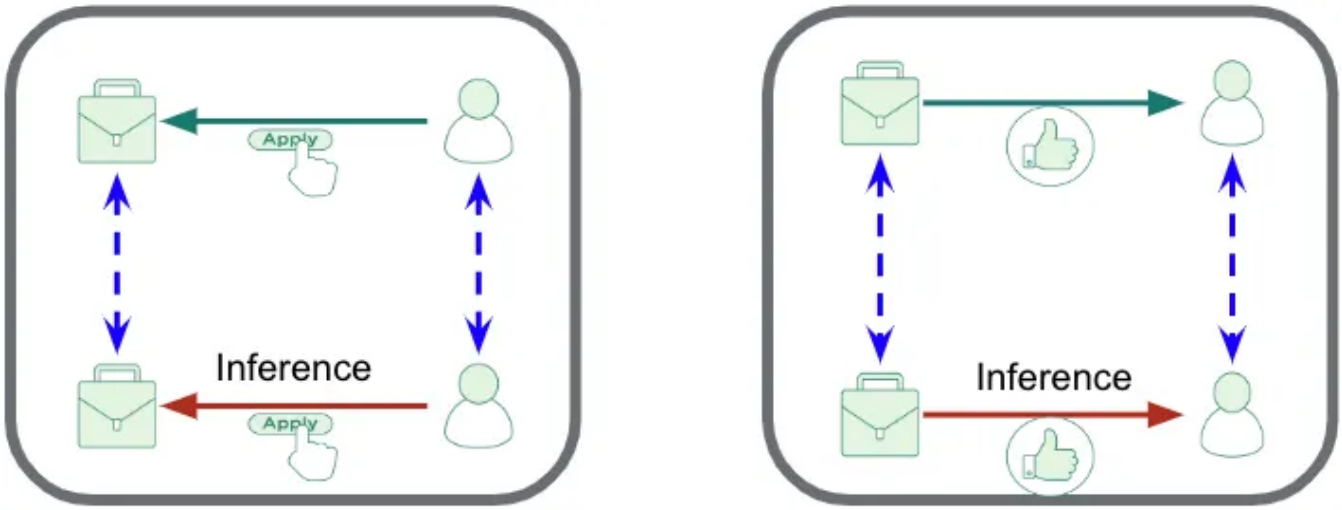


Figure 5: Cross-entity inference tries to utilize the learned latent representations to infer the similarity between two unseen different entities.

Since the ultimate goal is to predict which jobs are most suited for a given job-seeker, we need to infer cross-entity similarity. This kind of similarity metric learning is difficult to optimize directly within a single modeling architecture.

After understanding the need for pre-trained encoders and how they are built for imposing the required semantic similarity we can proceed to define their specific structure.

**Job Encoder**

The purpose of this encoder is to model the similarity of two distinct jobs based on the fact that a given job-seeker has applied to both of them. We chose to model similar jobs using a **siamese network in which the optimization itself is based on a contrastive focal loss.**

Get Engineering Team's stories in your inbox

Join Medium for free to get updates from this writer.

Subscribe

Remember me for faster sign in

In practice, we collect job pairs that were co-applied by the same job-seeker. During model training, these pairs serve as the positive examples in the data. Figure 6 depicts the architecture of the model where the objective is to learn the parameters of the encoder function $f$ -

$$\mathcal{M}_{\mathcal{V}} = f(\mathcal{V}), \quad \mathcal{M}_{\mathcal{V}} \in \mathcal{R}^d$$

Using a regular softmax-based contrastive loss may result in a very small value for the temperature parameter which represents a too narrow probability distribution, eventually leading to overconfident predictions. This means that due to the dominance of easy examples, the model would only learn the portion of easy pairs in the data. Conversely, the advantage of a ***focal loss*** formulation is that it prevents such a scenario and provides more weight to the hard misclassified pairs in the data.

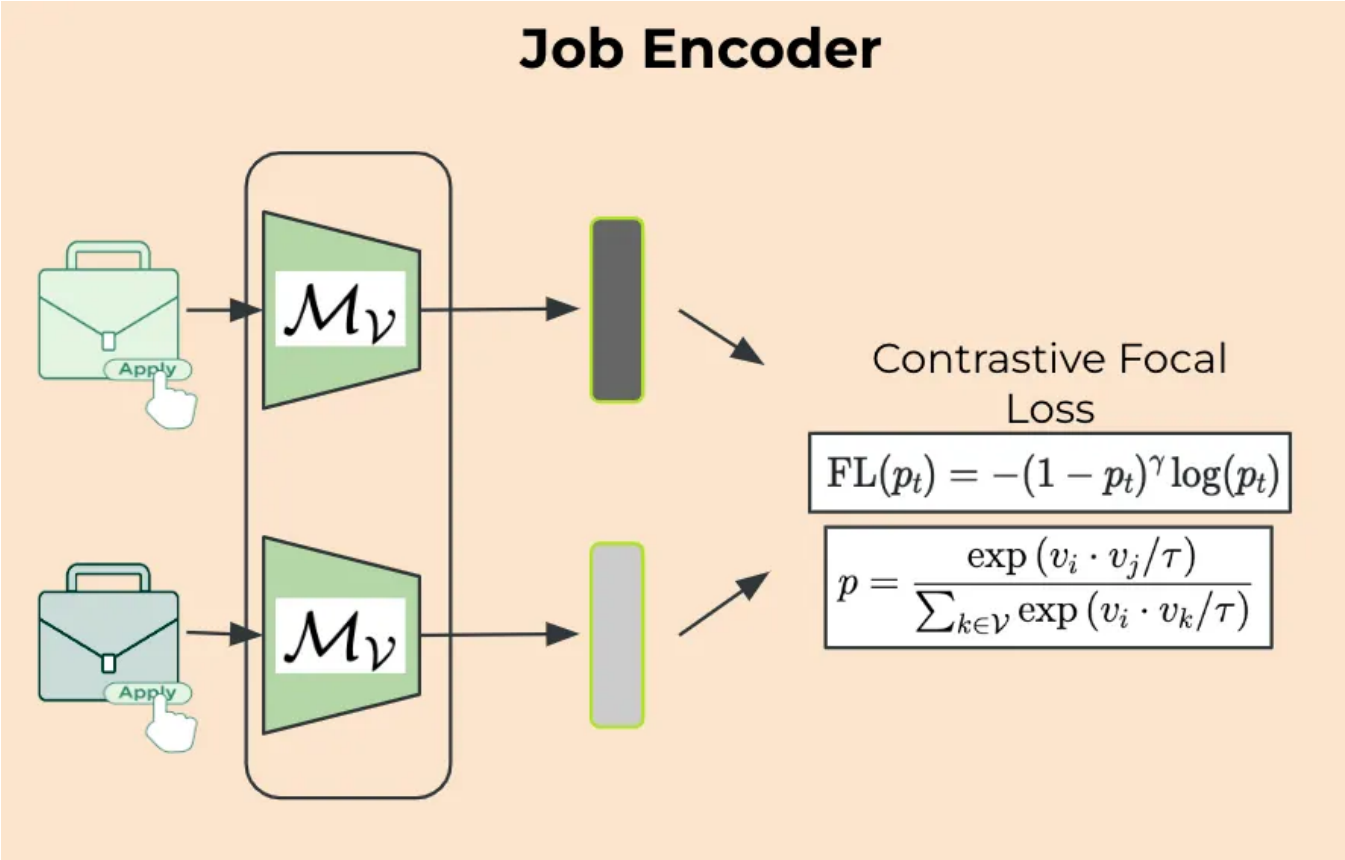


Figure 6: Using the standard contrastive loss might learn the ratio of easy pairs in the data. Combining it with focal loss enables the model to focus on the more interesting pairs of co-applications.

**Resume Encoder**

Similar considerations are applied for learning the job-seeker's resume encoder. There is, however, one major difference. In this scenario, the informative signal is generated by the employer who posted the job. If the employer finds the job-seeker relevant for the job they can provide positive feedback by selecting a thumbs-up (👍) indicating that the employer may be interested in initiating a hiring process with this candidate. On the other hand, if the job-seeker is not relevant for the job, the employer might provide a negative signal using a thumbs-down (👎).

A natural approach is to use a triplet loss function. This loss learns a similarity metric by enforcing the distance between a pair of similar job-seekers to be smaller than a pair of job-seekers that are not similar. In our framework, similar corresponds to a pair of job-seekers who received a thumbs-up from a specific employer on a certain job while non-similar corresponds to a pair of job-seekers in which one of them got a thumbs-up while the other got a thumbs-down. In addition,

since most jobs have more thumbs-down than thumbs-up, we are able to enrich the training data with multiple negative samples associated with every positive sample.

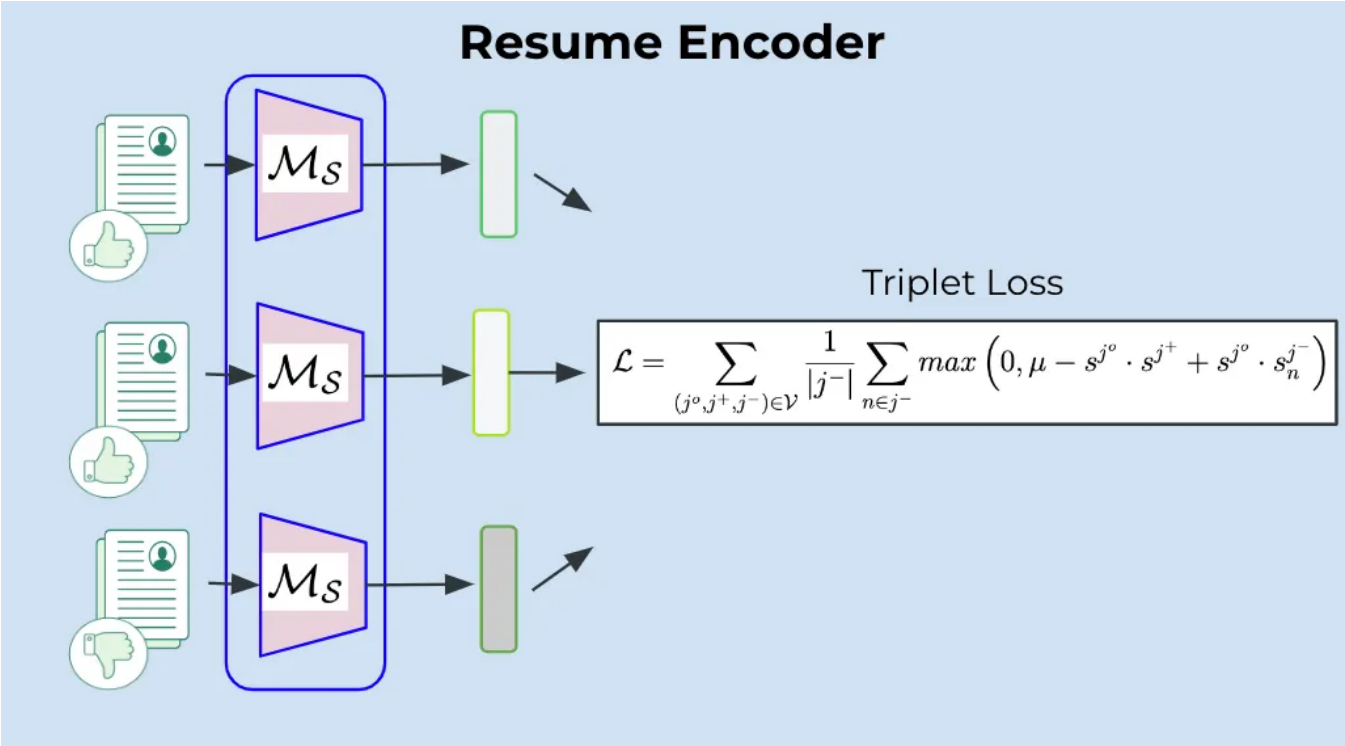


Figure 7: A triplet loss function enforces proximity between job seekers for a specific job who have both received a thumbs-up from the employer.

## Learning to infer across entities — the modeling architecture

So far, we've described how we are able to model similarity within the same entity type, i.e. job-to-job and job-seeker-to-job-seeker, which allows the representation and even predict relations between unseen entities. In order to model the cross-entity semantic relation between job-seekers and jobs, we now need to combine the relevant pieces and establish the mechanism for learning the similarity between different entity types.

This mechanism should have two central components that will represent the two marketplace modalities. For the jobs modality, the representation of a job remains static throughout the job's life and does not depend on any interaction occurring in the marketplace. As such, we can already make use of our existing pre-trained job

encoder.

For the job-seeker modality, the formulation is less trivial. To understand better, let's recall that our objective is to build a job recommendation model for job-seekers. This means that a similarity score between a job-seeker and a job should designate how relevant the job is to that specific job-seeker. Although the word 'relevance' in this context may require a stricter definition, we can assume that it is implicitly indicated as **the willingness of the job-seeker to engage with this job**. In light of that, having only the resume representation of a job-seeker is not enough and we will need to construct a model that will encompass all the available information we have about the job-seeker and produce a complete job-seeker representation. This may include contextual information like the resume and relevant engagement data like the searches, applications, employer's interest signals and more.

### Model Architecture

Our chosen architecture needs to incorporate both static resume data as well as the job-seeker's time-dependent recent activity. As described above, the resume is represented by a pre-trained encoder, however, processing the job-seeker activity is slightly more complex.

Modeling time-dependent data is usually done by employing an appropriate sequence aware architecture, like RNN or transformer. In our case, the job-seeker's sequence consists of not only explicit job interactions (e.g. clicks, applications) but also searches conducted on ZipRecruiter's homepage or mobile application. For job interactions we can leverage the fact that we have a consistent and independent way of representing jobs. Thus, using the pre-trained job encoder we can feed our model with the relevant embedding vectors along with a dedicated encoding designating the distinct interaction type -

$$job\ interaction = [\underbrace{e_1 e_2 \cdots e_d}_{job\ embedding},\ \underbrace{1\ \ \cdots\ \ 0}_{Action\ OHE}]$$

Transforming searches into suitable embedding vectors naturally requires a different approach. In a search, a job-seeker has entered free text, often without

grammatical structure, which represents specific intentions. These searches are accompanied by a location specifying where the job-seeker is looking for a job.

However, since searches do not have clear correspondence to actual jobs, constructing a pre-trained encoder is not relevant. Hence, a straightforward solution would be to simply build a dedicated search encoder which will be optimized as part of the full model and to match its output with the job encoder. In order to do that, we take advantage of the query structure which mostly consists of professional terms, like desired job titles or industries, relevant skills set or even abstract job characteristics, like 'remote', 'entry level', etc.

We start by mapping the search query to a set of meaningful terms by extracting and standardizing them according to a given taxonomy. Then, we calculate a weighted average over the token dimension which yields a query level embedding representation. The weighted average is based on a modified self-attention mechanism -

$$e_{search} = \sum_i x_i \cdot \underbrace{softmax(x_i \cdot k^T)}_{w_i = per\ token\ weight},\ x \in \mathcal{R}^{tokens \times d},\ k \in \mathcal{R}^d$$

Now we are able to combine job-seekers' searches with their job interactions and maintain the structure of the sequence. In order to combine the job-seeker's past activity representation with their resume we use a feed-forward block to receive a unified single job-seeker embedding.

Our task is then to predict the next applied job by the job-seeker given their associated information. Recalling our efficiency considerations for using scalar product calculations over our massive available data, we can finally define the complete model architecture.

Given a job-seeker and job pair which are connected by an apply, we project the job-seeker representations onto the dual space of the job embedding space using their corresponding encoders and then calculate their scalar product. As the representations are L2 normalized, their dot product represents a similarity score

that is monotonic in the estimated probability of the job-seeker to apply for the job.

## Loss Function

Optimizing for similarity between pairs of entities naturally pushes us toward using a contrastive loss. This way, the model can learn to bring the representation of similar entities closer to one another and separate non-related pairs. Since we optimize for job-seekers, each one of them contributes a contrastive loss term of the form -

$$\mathcal{L}_{s_i} = -log\left(\frac{\exp\left(s_i \cdot v_i/\tau\right)}{\sum_{j\in\mathcal{V}}\exp\left(s_i \cdot v_j/\tau\right)}\right)$$

Where $i$ represents the $i$-th (job-seeker, job) pair indicating an application, $\mathcal{V}$ represents a set of sampled jobs, $s_i$ is a given job-seeker, $\tau$ represents a temperature parameter, and the dot represents a scalar multiplication between the embedding vectors.

Summing for all job-seekers in a batch we obtain

$$\mathcal{L}_{\mathcal{S}} = -\sum_{i\in\mathcal{S}_{batch}} log\left(\frac{\exp\left(s_i \cdot v_i/\tau\right)}{\sum_{j\in\mathcal{V}}\exp\left(s_i \cdot v_j/\tau\right)}\right)$$

where *S_batch* represents the set of job-seekers in the batch.

However, this loss form is mainly optimized for one side of the marketplace and, as we specified before, this might lead to undesired behavior of the model. Therefore, we need to balance our optimization process and account also for the jobs side of the marketplace.

Looking at the loss equation above, we can observe that the one-sided optimization

actually comes from the softmax normalization term. Here, for each job-seeker we sample jobs so the probability distribution is defined over the chosen jobs. This suggests that we should symmetrize the loss function and construct a similar loss term for the jobs side that will define a probability distribution over the job-seekers. Mathematically, this can be formulated by -

$$\mathcal{L}_{\mathcal{V}} = -\sum_{i \in \mathcal{V}_{batch}} log\left(\frac{\exp\left(v_i \cdot s_i/\tau\right)}{\sum_{j \in \mathcal{S}} \exp\left(v_i \cdot s_j/\tau\right)}\right)$$

Thus, we can define the dual-contrastive loss function which imposes that additional structure into the model -

$$\hat{\mathcal{L}} = \mathcal{L}_{\mathcal{S}} + \mathcal{L}_{\mathcal{V}}$$

Usually, building the contrastive loss pairs is based on the sample batch entities. First, we sample *n* positive pairs, that is, a job-seeker and a job that are connected by an actual application. Then, we can sample random pairs out of those n pairs and use them as the negative pairs. That way we can batch these pairs together in a simple and efficient manner for the model to use. Thus, we have *N* positive pairs and $(N^2 - N)$ random negative pairs:

| | | | | |
|---|---|---|---|---|
| $s_1 \cdot v_1$ | $s_1 \cdot v_2$ | $s_1 \cdot v_3$ | $\cdots$ | $s_1 \cdot v_N$ |
| $s_2 \cdot v_1$ | $s_2 \cdot v_2$ | $s_2 \cdot v_3$ | $\cdots$ | $s_2 \cdot v_N$ |
| $s_3 \cdot v_1$ | $s_3 \cdot v_2$ | $s_3 \cdot v_3$ | $\cdots$ | $s_3 \cdot v_N$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ |
| $s_N \cdot v_1$ | $s_N \cdot v_2$ | $s_N \cdot v_3$ | $\cdots$ | $s_N \cdot v_N$ |

Figure 8: Matrix representation of a training batch. The diagonal designates the positive pairs.

In a matrix form, each row corresponds to a single job-seeker loss term where the *(i, i)* pair is the numerator while the entire row composes the normalization term. Therefore, summing rows produces the complete loss ($\mathcal{L}_s$). Following the same logic for the columns yields the Jobs loss ($\mathcal{L}_v$).

Examining the job-seeker loss, we observe that the batch may include duplicate jobs which effectively decrease the coverage jobs and affect the estimated distribution. Therefore, we can reconsider the normalization term and instead of being restricted to the jobs that appear in the batch we can exploit the fact that jobs are represented by a pre-trained encoder and uniformly sample as many jobs as we like from the

available job corpus. Note that such a sample differs from in-batch sampling which follows the training set job distribution governed by more interacted jobs. This corpus reflects the potential space of jobs, i.e.:

$$\mathcal{L}_{\mathcal{S}} = -\sum_{i \in \mathcal{S}_{batch}} log\left(\frac{\exp\left(s_i \cdot v_i/\tau\right)}{\sum_{j \in \mathcal{V}_{corpus}} \exp\left(s_i \cdot v_j/\tau\right)}\right)$$

This consideration, however, is not applicable for the second loss. This is because sampling job-seekers from a corpus requires us to perform inference for every such job-seeker, in addition to the job-seekers already in the batch, which might significantly affect the model performance.

In practice, in order to increase learning capacity and reduce bias, in each iteration, we build two similarity matrices and simply sample the corresponding negative pairs independently for each task. For suggesting jobs to job-seekers we sample from the entire job corpus, whereas for the opposite recommendation direction we sample them directly from the batch.

Here, the suggested job loss has a straightforward probabilistic interpretation which is the probability that a job-seeker will apply for the job while the second loss term has the probabilistic interpretation of the probability that a given job will be applied to by a job-seeker. This formulation essentially makes the training more stable and leads to better model performance.

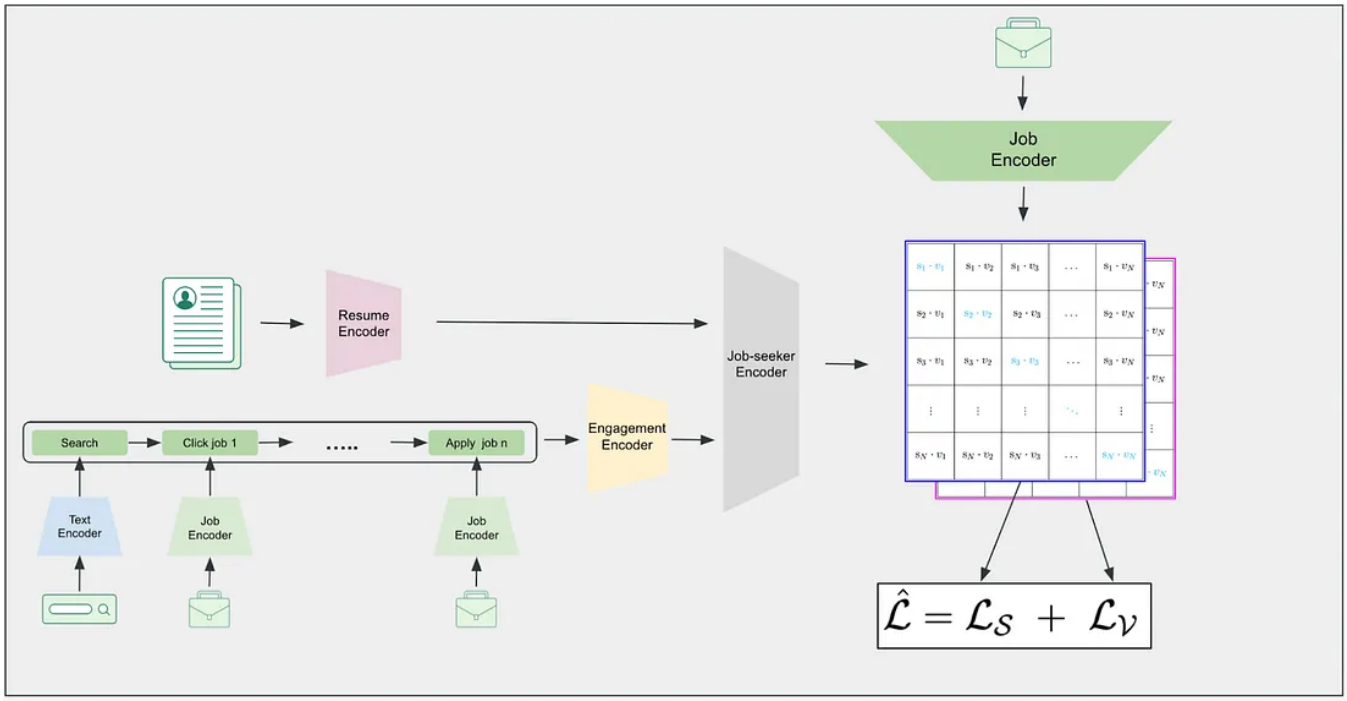


Figure 9: Bringing all components together for a complete modeling system.

## Results and Future Development

After building the first version of the entire modeling framework we A/B tested it on several recommendation applications facing job-seekers. The results showed a significant improvement in compute performance, an overall 12% lift in job-seeker engagement, and over 18% increase in satisfaction.

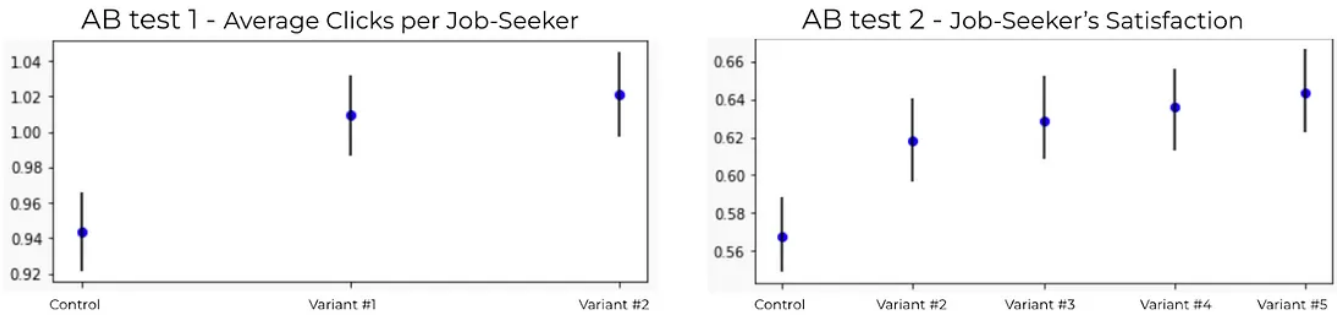


Figure 10: AB tests results. The new model variants demonstrate clear lift in performance.

We continue to develop this model, and an advanced and improved version

currently powers the recommendations for many of ZipRecruiter's core products.

Going forward, we are developing a graph neural networks (GNN) model that will serve as the base modeling framework for recommendations. Ideally, using a GNN we will be able to simultaneously perform link prediction between different entity types within a single embedding space and model all these similarities and attributes together. This will open the door to including more signals such as how employers acted upon suggestions from our platform regarding job-seekers.

***If you're interested in working on solutions like these, visit our Careers page to see open roles.***

**About the author:**

Nadav Barkai is a Data Science and Engineering manager on ZipRecruiter's Recommendation's team. Leveraging his vast background in physics and complex algorithm development, he leads a mixed team of data scientists and engineers in building the matching and recommendation models at the heart of ZipRecruiter's smart employment marketplace.

Software Engineer | Data Science | Recommendation System | Machine Learning | AI

Follow

## Published in ZipRecruiter Tech

12 followers · Last published Oct 3, 2023

Here, you'll find articles written by our R&D and Product teams. We aim to provide you with a glimpse into the technology and the people behind ZipRecruiter. Join our mission to actively connect people to their next great opportunity.

Follow

## Written by Engineering Team

121 followers · 10 following

The engineering team at ZipRecruiter®

## No responses yet

Write a response

What are your thoughts?